

NAME : DISHA SAMANTA **CUID :** CU24250227 **ROLLNO :** 20

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

from google.colab import files

print("Libraries imported successfully")
```

Libraries imported successfully

```
uploaded = files.upload()

video_filename = list(uploaded.keys())[0]

print("Video uploaded successfully:")
print(video_filename)
```

No file chosen

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving 13646619_1080_1920_30fps.mp4 to 13646619_1080_1920_30fps.mp4

Video uploaded successfully:
13646619_1080_1920_30fps.mp4

```
cap = cv2.VideoCapture(video_filename)

if not cap.isOpened():
    raise SystemExit("Unable to open video.")

ret, first_frame = cap.read()

if not ret:
    cap.release()
    raise SystemExit("Unable to read video.")

gray_first = cv2.cvtColor(
    first_frame,
    cv2.COLOR_BGR2GRAY
)

print("First frame read successfully.")
```

First frame read successfully.

```
feature_params = dict(
    maxCorners=100,
    qualityLevel=0.3,
    minDistance=7,
    blockSize=7
```

```
)

p0 = cv2.goodFeaturesToTrack(
    gray_first,
    mask=None,
    **feature_params
)

if p0 is None:
    cap.release()
    raise SystemExit("No feature points detected.")

print("Feature points detected:", len(p0))

shi_tomasi_frame = first_frame.copy()

for point in p0:
    x, y = point.ravel().astype(int)

    cv2.circle(
        shi_tomasi_frame,
        (x, y),
        4,
        (0, 255, 0),
        -1
    )

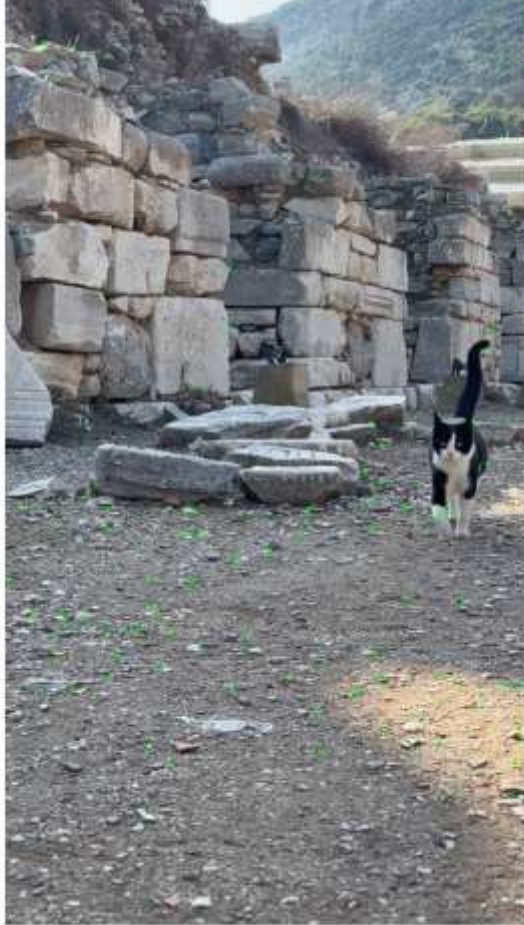
plt.figure(figsize=(8, 6))

plt.imshow(
    cv2.cvtColor(
        shi_tomasi_frame,
        cv2.COLOR_BGR2RGB
    )
)

plt.title("Output 1: Shi-Tomasi Feature Points")
plt.axis("off")
plt.show()
```

Feature points detected: 100

Output 1: Shi-Tomasi Feature Points

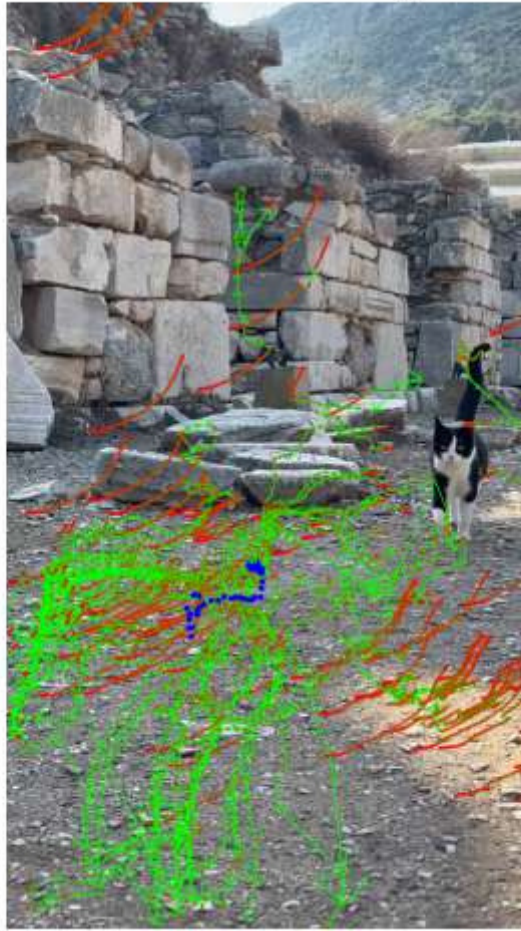


```
plt.figure(figsize=(8, 6))

plt.imshow(
    cv2.cvtColor(
        trajectory_frame,
        cv2.COLOR_BGR2RGB
    )
)

plt.title("Output 2: Lucas-Kanade Tracking and Trajectory")
plt.axis("off")
plt.show()
```

Output 2: Lucas-Kanade Tracking and Trajectory



```
if frame_count > 0:

    average_displacement = (
        total_displacement / frame_count
    )

    average_speed = (
        average_displacement * fps
    )

    dx = current_center[0] - previous_center[0]
    dy = current_center[1] - previous_center[1]

    if abs(dx) > abs(dy):

        if dx > 0:
            direction = "Right"
        else:
            direction = "Left"

    else:

        if dy > 0:
```

```
        direction = "Down"
    else:
        direction = "Up"

else:

    average_displacement = 0
    average_speed = 0
    direction = "No Motion"

print("Average Displacement:", average_displacement)
print("Average Speed:", average_speed)
print("Motion Direction:", direction)
```

Average Displacement: 5.532279
Average Speed: 165.96837
Motion Direction: Up

```
cap = cv2.VideoCapture(video_filename)

ret, frame1 = cap.read()
ret, frame2 = cap.read()

if not ret:
    cap.release()
    raise SystemExit("Unable to read consecutive frames.")

gray1 = cv2.cvtColor(
    frame1,
    cv2.COLOR_BGR2GRAY
)

gray2 = cv2.cvtColor(
    frame2,
    cv2.COLOR_BGR2GRAY
)

print("Two consecutive frames ready.")
```

Two consecutive frames ready.

```
flow = cv2.calcOpticalFlowFarneback(
    gray1,
    gray2,
    None,
    0.5,
    3,
    15,
    3,
    5,
```

```
        1.2,  
        0  
    )  
  
    magnitude, angle = cv2.cartToPolar(  
        flow[..., 0],  
        flow[..., 1]  
    )  
  
    average_flow_magnitude = np.mean(magnitude)  
  
    print(  
        "Average Farneback Flow Magnitude:",  
        average_flow_magnitude  
    )
```

Average Farneback Flow Magnitude: 0.82715327

```
# Output 3: Consecutive Frame  
  
plt.figure(figsize=(8, 6))  
  
plt.imshow(  
    cv2.cvtColor(  
        frame2,  
        cv2.COLOR_BGR2RGB  
    )  
)  
  
plt.title("Output 3: Consecutive Video Frame")  
plt.axis("off")  
plt.show()  
  
# Output 4: Farneback Dense Optical Flow  
  
plt.figure(figsize=(8, 6))  
  
plt.imshow(  
    cv2.cvtColor(  
        dense_flow_frame,  
        cv2.COLOR_BGR2RGB  
    )  
)  
  
plt.title("Output 4: Farneback Dense Optical Flow")  
plt.axis("off")  
plt.show()  
  
cap.release()
```